

HCI & CG ASSIGNMENT

Name: Muhammad Mudassir

Roll no: 2K24/CSE/101

Submitted to : sir rajesh kumar

Task1: Display Pixel Density (PPI/DPI) Calculator

Code:

```
import math
# Take screen information from the user
width_pixels = int(input("Enter horizontal resolution (pixels): "))
height_pixels = int(input("Enter vertical resolution (pixels): "))
diagonal_inches = float(input("Enter physical diagonal size (inches): "))

# Validate the input
if width_pixels <= 0 or height_pixels <= 0 or diagonal_inches <= 0:
    print("Error: All values must be greater than zero.")
else:
    # Calculate total pixels
    total_pixels = width_pixels * height_pixels

    # Calculate simplified aspect ratio
    common_divisor = math.gcd(width_pixels, height_pixels)
    aspect_width = width_pixels // common_divisor
    aspect_height = height_pixels // common_divisor

    # Calculate diagonal pixels and PPI
    diagonal_pixels = math.sqrt(
        width_pixels ** 2 + height_pixels ** 2
    )

    ppi = diagonal_pixels / diagonal_inches

    # Classify screen density
    if ppi < 100:
        density_category = "Low Density (Standard Monitor)"
    elif ppi <= 200:
        density_category = "Medium Density (HD Display)"
    else:
        density_category = "High Density (Retina / Mobile)"

    # Display results
    print("\n--- DISPLAY METRICS ANALYSIS ---")
    print(f"Total Pixel Count : {total_pixels:,} pixels")
    print(f"Aspect Ratio       : {aspect_width}:{aspect_height}")
    print(f"Calculated DPI      : {ppi:.2f} DPI")
    print(f"Density Category   : {density_category}")
```

output 1:

```
PS D:\Desktop\HCI lab> py task1_display_metrics.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
PS D:\Desktop\HCI lab>
```

output 2:

```
PS D:\Desktop\HCI lab> py task_display_metrics.py
PS D:\Desktop\HCI lab> py task1_display_metrics.py
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
PS D:\Desktop\HCI lab>
```

Task2: *Environment Setup & Synthetic Image Matrix Creation*

Code:

```
import numpy as np
from PIL import Image

# Image dimensions
height = 300
width = 400
channels = 3

# Create a black RGB image containing zeros
image_array = np.zeros(
```

```

    (height, width, channels),
    dtype=np.uint8
)

# Find the halfway points
half_height = height // 2
half_width = width // 2

# Fill the four quadrants

# Top-left: Red
image_array[0:half_height, 0:half_width] = [255, 0, 0]

# Top-right: Green
image_array[0:half_height, half_width:width] = [0, 255, 0]

# Bottom-left: Blue
image_array[half_height:height, 0:half_width] = [0, 0, 255]

# Bottom-right: White
image_array[half_height:height, half_width:width] = [255, 255, 255]

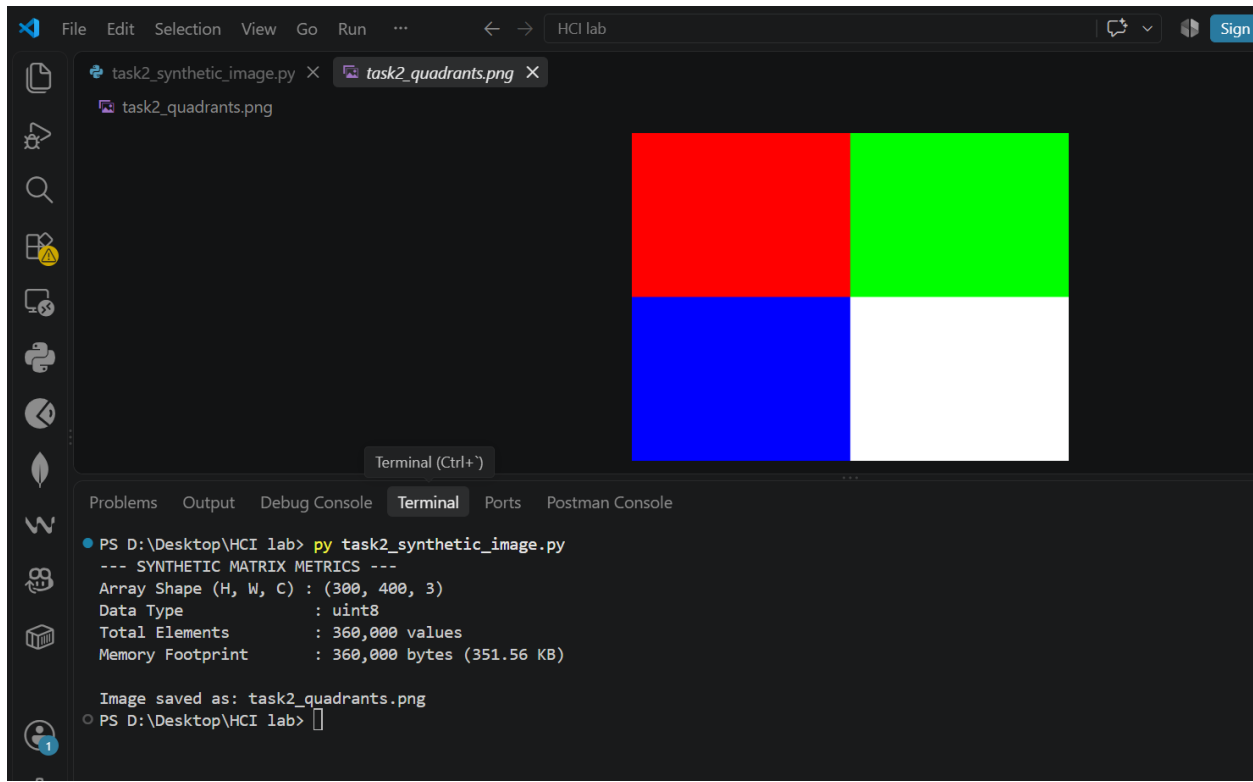
# Print matrix information
print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {image_array.shape}")
print(f>Data Type           : {image_array.dtype}")
print(f"Total Elements      : {image_array.size:,} values")
print(
    f"Memory Footprint      : {image_array.nbytes:,} bytes "
    f"({image_array.nbytes / 1024:.2f} KB)"
)

# Save the synthetic image
output_image = Image.fromarray(image_array)
output_image.save("task2_quadrants.png")

print("\nImage saved as: task2_quadrants.png")

```

output:



Task3: Channel Slicing & Isolation

Code:

```
import numpy as np  
import matplotlib.pyplot as plt  
from PIL import Image  
  
# Load the image and force it into RGB format  
pil_image = Image.open("sample.jpg").convert("RGB")  
  
# Convert the image into a NumPy array  
image = np.array(pil_image)  
  
# Extract the three 2D channel intensity grids  
red_channel = image[:, :, 0]  
green_channel = image[:, :, 1]  
blue_channel = image[:, :, 2]
```

```

# Create three empty RGB arrays
red_only = np.zeros_like(image)
green_only = np.zeros_like(image)
blue_only = np.zeros_like(image)

# Activate only the required channel
red_only[:, :, 0] = red_channel
green_only[:, :, 1] = green_channel
blue_only[:, :, 2] = blue_channel

# Print channel information
print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape : {image.shape}")
print(
    f"Red Channel 2D Shape : {red_channel.shape} | "
    f"Mean Intensity: {red_channel.mean():.2f}"
)
print(
    f"Green Channel 2D Shape: {green_channel.shape} | "
    f"Mean Intensity: {green_channel.mean():.2f}"
)
print(
    f"Blue Channel 2D Shape : {blue_channel.shape} | "
    f"Mean Intensity: {blue_channel.mean():.2f}"
)

# Create a 2 x 3 subplot
figure, axes = plt.subplots(2, 3, figsize=(14, 8))

# Top row: isolated color images
axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only Image")

axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only Image")

axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only Image")

# Bottom row: grayscale intensity maps
axes[1, 0].imshow(red_channel, cmap="gray", vmin=0, vmax=255)
axes[1, 0].set_title("Red Channel Intensity")

axes[1, 1].imshow(green_channel, cmap="gray", vmin=0, vmax=255)
axes[1, 1].set_title("Green Channel Intensity")

```

```
axes[1, 2].imshow(blue_channel, cmap="gray", vmin=0, vmax=255)
axes[1, 2].set_title("Blue Channel Intensity")

# Remove axis numbers
for axis in axes.flat:
    axis.axis("off")

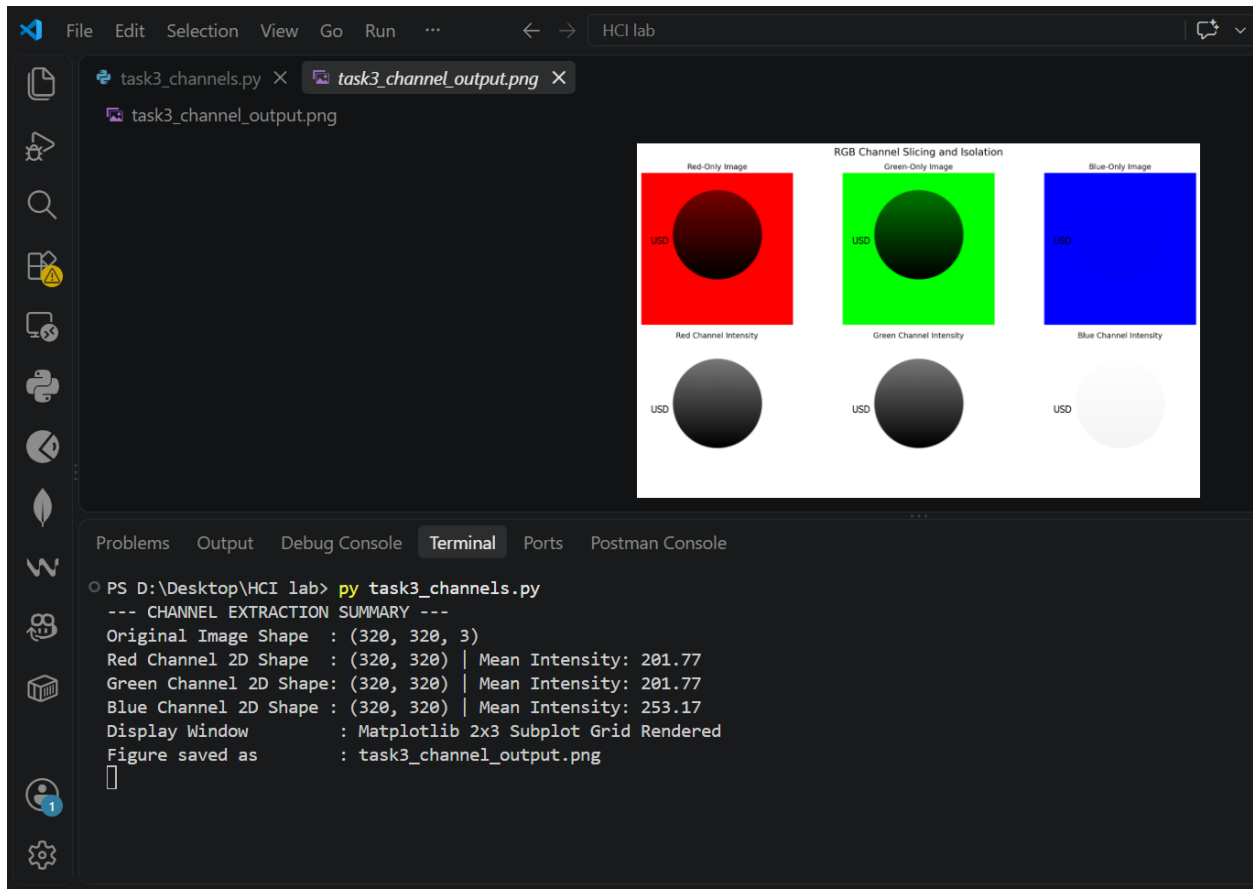
plt.suptitle("RGB Channel Slicing and Isolation", fontsize=16)
plt.tight_layout()

# Save the figure for the report
plt.savefig(
    "task3_channel_output.png",
    dpi=200,
    bbox_inches="tight"
)

print("Display Window          : Matplotlib 2x3 Subplot Grid Rendered")
print("Figure saved as         : task3_channel_output.png")

plt.show()
```

output:



Task 4: patial Downsampling & Pixelation via Striding

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load the original image
pil_image = Image.open("sample.jpg").convert("RGB")
original_image = np.array(pil_image)

# Downsampling step factor
N = 8

# Take every 8th row and every 8th column
downsampled_image = original_image[::N, ::N, :]
```

```

# Re-expand the image by repeating pixels
pixelated_image = np.repeat(
    downsampled_image,
    N,
    axis=0
)

pixelated_image = np.repeat(
    pixelated_image,
    N,
    axis=1
)

# Crop it to the exact original dimensions
original_height = original_image.shape[0]
original_width = original_image.shape[1]

pixelated_image = pixelated_image[
    :original_height,
    :original_width,
    :
]

# Calculate dimension reductions
height_reduction = (
    1 - downsampled_image.shape[0] / original_image.shape[0]
) * 100

width_reduction = (
    1 - downsampled_image.shape[1] / original_image.shape[1]
) * 100

# Calculate memory savings
memory_savings = (
    1 - downsampled_image.nbytes / original_image.nbytes
) * 100

# Print results
print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")

print(
    f"Original Shape      : {original_image.shape} | "
    f"Memory: {original_image.nbytes:,} bytes"
)

```

```

print(
    f"Downsampled Shape : {downsampled_image.shape} | "
    f"Memory: {downsampled_image.nbytes:,} bytes"
)

print(
    f"Re-expanded Shape : {pixelated_image.shape} | "
    f"Visual: Blocky Pixelation"
)

print(f"Height Reduction : {height_reduction:.2f}%")
print(f"Width Reduction : {width_reduction:.2f}%")
print(f"Memory Savings : {memory_savings:.2f}% data reduction")

# Display the results
figure, axes = plt.subplots(1, 3, figsize=(16, 6))

axes[0].imshow(original_image)
axes[0].set_title("Original Image")

axes[1].imshow(downsampled_image)
axes[1].set_title(f"Downsampled Image (N = {N})")

axes[2].imshow(pixelated_image)
axes[2].set_title("Re-expanded Pixelated Image")

for axis in axes:
    axis.axis("off")

plt.suptitle("Spatial Downsampling Using NumPy Striding", fontsize=16)
plt.tight_layout()

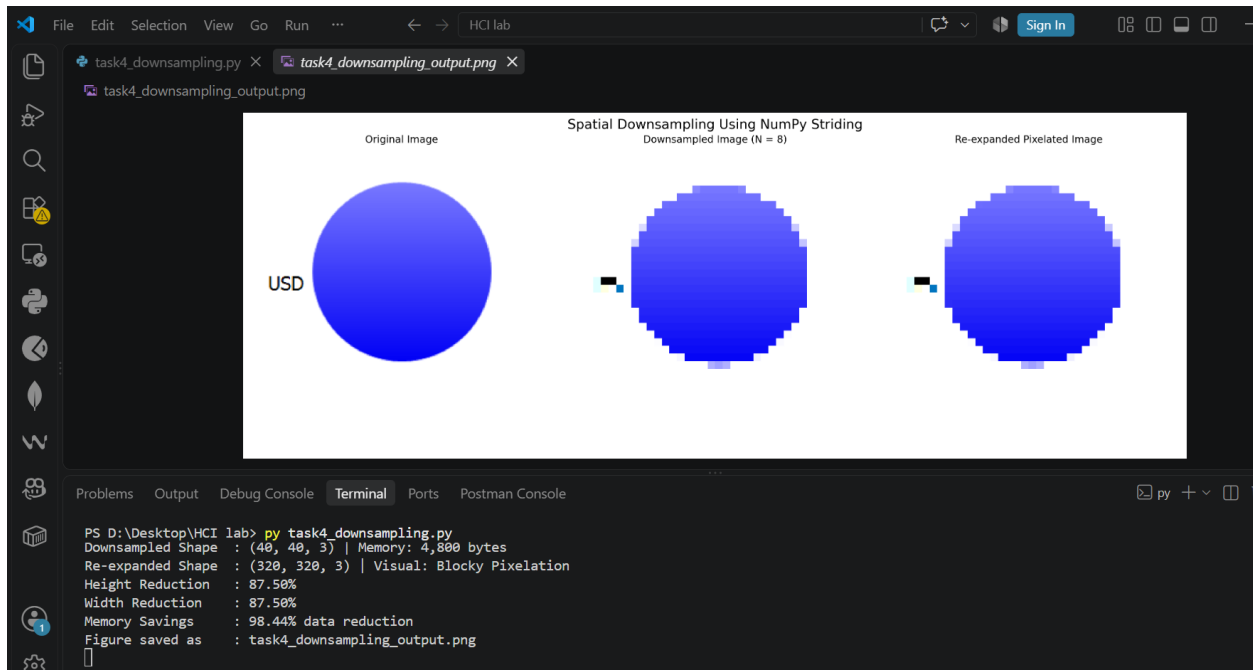
# Save the figure
plt.savefig(
    "task4_downsampling_output.png",
    dpi=200,
    bbox_inches="tight"
)

print("Figure saved as : task4_downsampling_output.png")

plt.show()

```

output:



Short Answer:

An RGB image uses three channels: Red, Green, and Blue. An RGBA image uses four channels because it adds an Alpha channel for transparency. If every channel uses one byte per pixel, RGB requires 3 bytes per pixel while RGBA requires 4 bytes. Therefore, the increase is $(4 - 3) \div 3 \times 100 = 33.33\%$. Thus, RGBA consumes approximately 33% more memory than RGB for an image with the same dimensions and data type.